

Sommario

Importazioni.....	1
<i>S3 Client</i>	1
📦 createBucket	1
🗑 deleteBucket	2
? doesBucketExist	2
📦 listBuckets	2
📦 listObjects	3
📦 getIterator (variante di listObjects)	3
📦 putObject	4
📦 getObject	5
🔗 getObjectUrl	5
🗑 deleteObject.....	6

<i>DynamoDB Client</i>	1
📦 createTable	1
🗑 deleteTable.....	3
📄 listTables.....	3
🔄 updateTable.....	4
📄 describeTable	5
🔍 scan	6
📦 putItem	7
🔄 updateItem	8
🔍 getItem	9
🔍 query	10
🗑 deleteItem	11

Importazioni

Per poter usare AWS SDK in PHP, è necessario includere il file di autoload e importare le classi utili:

```
require 'vendor/autoload.php';
use Aws\S3\S3Client;
use Aws\Exception\AwsException;
```

🔑 Accesso con credenziali

Le credenziali si possono specificare direttamente nel codice o, preferibilmente, caricare da un file di configurazione (es. config.php). Esempio con inserimento diretto (non consigliato in produzione):

```
$credentials = new
Aws\Credentials\Credentials('chiave',
'segreto');
```

S3 Client

Per interagire con il servizio S3 si utilizza S3Client, specificando almeno:

- version: di default **"latest"**
- region: es. **"eu-south-1"**
- credentials: oggetto **Credentials**

```
$s3 = new Aws\S3\S3Client([
    'version' => 'latest',
    'region' => 'eu-south-1',
    'credentials' => $credentials
]);
```

📁 Metodi principali

📦 createBucket

• Crea un nuovo **bucket S3** specificando il nome e, opzionalmente, la regione. In molte regioni, come eu-south-1, è **obbligatorio** fornire l'opzione LocationConstraint altrimenti la creazione fallisce.

✳ Sintassi:

```
$result = $s3->createBucket([
    'Bucket' => '<nome-bucket>', //
    obbligatorio
    'CreateBucketConfiguration' => [
        'LocationConstraint' => '<regione>'
    ]
]);
```

📄 Descrizione dei parametri

Parametro	Descrizione
Bucket	(Obbligatorio) Nome del bucket da creare. Deve essere univoco a livello globale.
CreateBucket-Configuration	Oggetto opzionale richiesto in molte regioni per specificare la regione del bucket.
LocationConstraint	Regione AWS dove verrà creato il bucket (es. eu-south-1 per Milano).

⚠ Se non specifichi LocationConstraint, il bucket verrà creato nella regione di default del client, ma **in alcune regioni dà errore**.

💡 Esempio:

```
$bucket = 'nomedelbucket';

$result = $s3->createBucket([
    'Bucket' => $bucket,
    'CreateBucketConfiguration' => [
        'LocationConstraint'=> 'eu-south-1'
    ]
]);
```

⌚ Attendere la creazione effettiva del bucket

Dopo aver inviato la richiesta, si consiglia di **attendere la conferma effettiva** prima di eseguire operazioni sul bucket.

Puoi usare waitUntil per bloccare finché il bucket non è disponibile:

```
$s3->waitUntil('BucketExists', [
    'Bucket' => $bucket
]);
```

deleteBucket

- Elimina in modo permanente un bucket S3.
- ⚠ Il bucket **deve essere vuoto** prima della rimozione, altrimenti la chiamata fallisce con un errore.

✳ Sintassi:

```
$result = $s3->deleteBucket([
    'Bucket' => '<nome-bucket>' /*
obbligatorio*/
]);
```

Descrizione dei parametri

Parametro	Descrizione
Bucket	(Obbligatorio) Nome del bucket da eliminare.

💡 Esempio:

```
$s3->deleteBucket([
    'Bucket' => 'archivio-vecchio'
]);
```

📁 Questo comando rimuove il bucket chiamato archivio-vecchio, **se e solo se è vuoto**.

📄 Output (semplificato):

```
[
    'RequestId' => '<string>',
    'DeleteMarker' => true|false,
]
```

Descrizione dei parametri dell'output

Parametro	Descrizione
RequestId	ID univoco della richiesta, utile per il tracciamento lato AWS.
DeleteMarker	Presente solo se il versionamento è attivo. Indica se è stato creato un "delete marker".

? doesBucketExist

- Verifica **se un bucket esiste** nel servizio S3. Non distingue se il bucket è di proprietà dell'utente oppure no: restituisce true anche se il bucket è privato e non accessibile.

✳ Sintassi:

```
$esiste = $s3->doesBucketExist('<nome-
bucket>');
```

Descrizione dei parametri

Parametro	Descrizione
<nome-bucket>	(Obbligatorio) Nome del bucket da verificare.

⚠ **Attenzione:** Se il bucket esiste ma **non è di proprietà dell'utente**, non sarà possibile interagirci (es. putObject fallirà comunque).

💡 Esempio:

```
$bucket = 'nomedelbucket';

if (! $s3->doesBucketExist($bucket)) {
    $res = $s3->createBucket([
        'Bucket' => $bucket,
        'LocationConstraint' => 'eu-south-1'
    ]);

    $s3->waitUntil('BucketExists',
        ['Bucket' => $bucket]);
}
```

listBuckets

- Recupera l'elenco dei bucket appartenenti all'account.

Sintassi:

```
$result = $s3->listBuckets();
```

Output (semplificato):

```
[
    'Buckets' => [
        [
            'Name' => '<string>',
            'CreationDate' => <DateTime>
        ],
        ...
    ],
    'Owner' => [
        'DisplayName' => '<string>',
        'ID' => '<string>'
    ]
]
```

Parametri:

Nessuno richiesto.

listObjects

- Elenca gli oggetti contenuti in un bucket.

Sintassi:

```
$result = $s3->listObjects([
    'Bucket' => '<string>', // obbligatorio
    'Prefix' => '<string>',
    'Delimiter' => '<string>',
    'Marker' => '<string>',
    'MaxKeys' => <int>,
    'EncodingType' => 'url',
    'RequestPayer' => 'requester',
]);
```

Descrizione dei parametri

Parametro	Descrizione
Bucket	(Obbligatorio) Nome del bucket.
Prefix	Limita la risposta agli oggetti che iniziano con il prefisso specificato.
Delimiter	Raggruppa le chiavi che contengono lo stesso prefisso. Utile per simulare directory.
Marker	Specifica il punto di inizio per l'elenco (usato per paginazione manuale).
MaxKeys	Numero massimo di oggetti da restituire (default: 1000).
EncodingType	Specifica il tipo di codifica delle chiavi restituite (es. 'url').
RequestPayer	Se impostato a 'requester', il requester (e non il bucket owner) paga la richiesta.

Esempio:

```
$elenco = $s3->listObjects([
    'Bucket' => $bucket['Name']
]);
```

Output (semplificato):

```
[
    'Contents' => [
        [
            'Key' => '<string>',
            'LastModified' => <DateTime>,
            'Size' => <int>,
            'StorageClass' => '<string>',
        ],
        ...
    ],
    'IsTruncated' => true|false,
    'Name' => '<string>',
    'Prefix' => '<string>',
    'MaxKeys' => <int>
]
```

getIterator (variante di listObjects)

- Restituisce un **iteratore automatico** per l'elenco degli oggetti in un bucket. Viene usato al posto di listObjects per **semplificare la gestione della paginazione**: l'iteratore effettua automaticamente più richieste per ottenere tutti gli oggetti disponibili, senza dover gestire IsTruncated e Marker manualmente.

Sintassi:

```
$iterator = $s3->getIterator('ListObjects', [
    'Bucket' => '<string>', // obbligatorio
    'Prefix' => '<string>',
    'Delimiter' => '<string>',
    'MaxKeys' => <int>,
    'EncodingType' => 'url',
]);
```

Descrizione dei parametri

Parametro	Descrizione
Bucket	(Obbligatorio) Nome del bucket.
Prefix	Limita l'elenco agli oggetti con prefisso specifico.
Delimiter	Raggruppa le chiavi simili (es. simulate directory).
MaxKeys	Numero massimo di oggetti per richiesta singola.
EncodingType	Codifica delle chiavi (es. 'url' per UTF-8 safe).

 L'iteratore gestisce internamente più chiamate HTTP a listObjects, fino a raccogliere tutti i risultati (paginazione automatica).

Esempio completo:

```
$result = $s3->listBuckets();
foreach ($result['Buckets'] as $bucket) {
    echo "<p>" . $bucket['Name'] . ": " .
    $bucket['CreationDate'] . "</p>\n";

    $iterator = $s3->getIterator(
        'ListObjects', [
            'Bucket' => $bucket['Name']
        ]
    );

    foreach ($iterator as $object) {
        echo $object['Key'] . "\n";
    }
}
```

✔ Vantaggi rispetto a listObjects

Aspetto	listObjects	getIterator
Gestione paginazione	Manuale (IsTruncated, Marker)	Automatica
Caricamento iniziale	Un batch per volta	Tutti i risultati disponibili
Performance	Controllo maggiore sull'uso	Maggiore semplicità d'uso
Uso consigliato	Quando serve controllo fine	Quando serve solo l'elenco completo

putObject

• Carica un file su un bucket specifico. Permette di specificare anche metadati personalizzati (es. tipo MIME) o usare direttamente il contenuto (Body) invece di un file locale (SourceFile).

✳ Sintassi:

```
$result = $s3->putObject([
    'Bucket' => '<string>', // obbligatorio
    'Key' => '<string>', // obbligatorio
    'SourceFile' => '<percorso_file>', /*
oppure 'Body' => '<contenuto>'/
    'Metadata' => [// opzionale
        'Content-Type' => '<string>'
    ],
]);
```

Descrizione dei parametri

Parametro	Descrizione
Bucket	(Obbligatorio) Nome del bucket.
Key	(Obbligatorio) Nome (chiave) con cui salvare l'oggetto nel bucket.
SourceFile	Percorso del file da caricare dal filesystem locale.
Body	Alternativa a SourceFile: contenuto grezzo da caricare.
Metadata	Dizionario con metadati personalizzati.

💡 Esempio completo:

```
$bucket = 'nomedelbucket';
$file = 'prova.jpg';
$result = $s3->putObject([
    'Bucket' => $bucket,
    'Key' => basename($file),
    'SourceFile' => $file,
    'Metadata' => [
        'Content-Type' => 'image/jpeg'
    ]
]);
print_r($result);
```

Output (semplificato)

```
[
    'ObjectURL' => '<string>',
    'ETag' => '<string>',
    'Expiration' => '<string|null>',
    'VersionId' => '<string|null>',
    'RequestId' => '<string>',
    'ServerSideEncryption' =>
    '<string|null>',
]
```

Descrizione dei parametri

Parametro	Descrizione
ObjectURL	URL completo dell'oggetto caricato.
ETag	Identificatore univoco dell'oggetto (di solito un hash MD5).
Expiration	Data di scadenza se è stata configurata una policy di lifecycle (opzionale).
VersionId	Identificativo di versione se il bucket ha versionamento attivo (opzionale).
RequestId	ID della richiesta generato da AWS per il tracciamento.
ServerSideEncryption	Metodo di cifratura utilizzato, se specificato (es. AES256, aws:kms).

getObject

• Scarica un oggetto (file) da un bucket S3. Restituisce un array con i metadati dell'oggetto e, se specificato, salva il contenuto in un file locale.

Sintassi:

```
$result = $s3->getObject([
    'Bucket' => '<nome-bucket>', /*
    obbligatorio */
    'Key' => '<nome-oggetto>', /*
    obbligatorio */
    'SaveAs' => '<percorso-locale>' /*
    opzionale: salva direttamente su disco */
]);
```

Descrizione dei parametri

Parametro	Descrizione
Bucket	(Obbligatorio) Nome del bucket.
Key	(Obbligatorio) Nome (chiave) del file da scaricare.
SaveAs	Percorso su disco dove salvare il file. Se omissso, il contenuto resta in memoria.

Esempio – Salvataggio su disco:

```
$result = $s3->getObject([
    'Bucket' => 'mio-bucket',
    'Key' => 'immagini/prova.jpg',
    'SaveAs' => 'download/prova.jpg'
]);
```

 Questo scaricherà il file prova.jpg dal bucket S3 e lo salverà localmente nella cartella download/.

Esempio – Accesso diretto al contenuto:

```
$result = $s3->getObject([
    'Bucket' => 'mio-bucket',
    'Key' => 'documenti/testo.txt'
]);
echo $result['Body']; /* stampa il
contenuto del file */
```

Output (semplificato):

```
[
    'Body' => <GuzzleHttp\Psr7\Stream>,
    'ContentLength' => <int>,
    'ContentType' => '<string>',
    'ETag' => '<string>',
    'LastModified' => <DateTime>,
    'Metadata' => [
        '<key>' => '<value>',
        ...
    ],
    'VersionId' => '<string>',
    'StorageClass' => '<string>',
    'AcceptRanges' => '<string>',
    'TagCount' => <int>,
]
```

Descrizione dei parametri

Parametro	Descrizione
Body	Flusso (stream) del contenuto dell'oggetto.
ContentLength	Lunghezza del contenuto in byte.
ContentType	Tipo MIME del contenuto.
ETag	Identificatore univoco dell'oggetto (hash MD5 tra virgolette).
LastModified	Data e ora dell'ultima modifica.
Metadata	Dizionario dei metadati personalizzati associati all'oggetto.
VersionId	ID della versione dell'oggetto (se versioning abilitato).
StorageClass	Classe di storage (es. STANDARD, INTELLIGENT_TIERING, ecc.).
AcceptRanges	Indica se sono supportate richieste parziali (es. "bytes").
TagCount	Numero di tag associati all'oggetto.

getObjectUrl

• Restituisce l'URL pubblico (o presunto tale) di un oggetto memorizzato in un bucket S3.
 **Non verifica l'esistenza effettiva dell'oggetto né la sua accessibilità pubblica.**

Sintassi:

```
$url = $s3->getObjectUrl(
    '<nome-bucket>', // obbligatorio
    '<nome-oggetto>', // obbligatorio
    '<timestamp scadenza opzionale>' /*
    opzionale */
);
```



Descrizione dei parametri

Parametro	Descrizione
bucket	(Obbligatorio) Nome del bucket.
key	(Obbligatorio) Nome (chiave) dell'oggetto nel bucket.
expires	(Opzionale) Timestamp Unix o stringa strtotime()-compatibile. Se fornito, genera un URL firmato con scadenza.



Esempio (URL pubblico presunto):

```
$url = $s3->getObjectUrl('archivio-dati',  
    'immagini/logo.png');
```



Restituisce l'URL **non firmato**, ad esempio:

`https://archivio-`

`dati.s3.amazonaws.com/immagini/logo.png`



Questo URL funzionerà solo se l'oggetto è

pubblicamente accessibile (es. via policy del bucket o ACL).



Esempio (URL firmato temporaneo):

```
$url = $s3->getObjectUrl('archivio-dati',  
    'immagini/logo.png', '+20  
    minutes');
```



Restituisce un URL con **firma e scadenza**, utile per l'accesso temporaneo.

L'oggetto può essere **privato**, ma accessibile tramite questo URL entro i 20 minuti.



Output:

```
(string) 'https://archivio-  
dati.s3.amazonaws.com/immagini/logo.png?...  
'
```



Note aggiuntive

- Non restituisce null o false se l'oggetto non esiste.
- Per controllare se l'oggetto esiste o è accessibile, usa `headObject`.
- Per URL firmati avanzati (con controllo sul metodo HTTP o altri header), usa `createPresignedRequest`.



deleteObject

- Elimina un **oggetto specifico** da un bucket S3.
- ⚠ Non elimina versioni precedenti se il versionamento è attivo.



Sintassi:

```
$result = $s3->deleteObject([  
    'Bucket' => '<nome-bucket>', /*  
    obbligatorio */  
    'Key'     => '<nome-oggetto>' /*  
    obbligatorio */  
]);
```



Descrizione dei parametri

Parametro	Descrizione
Bucket	(Obbligatorio) Nome del bucket.
Key	(Obbligatorio) Nome (chiave) dell'oggetto da eliminare.



Esempio:

```
$s3->deleteObject([  
    'Bucket' => 'archivio-dati',  
    'Key'     => 'report/2025/giugno.csv'  
]);
```



Questo comando elimina il file *giugno.csv* all'interno della cartella virtuale *report/2025*.



Output (semplificato):

```
[  
    'DeleteMarker' => true|false,  
    'VersionId'    => '<string>',  
    'RequestId'    => '<string>',  
]
```



Descrizione dei parametri dell'output

Parametro	Descrizione
DeleteMarker	(Solo con versionamento) True se è stato inserito un delete marker.
VersionId	(Solo con versionamento) ID della versione eliminata o marcata.
RequestId	ID della richiesta, utile per debug/log.

DynamoDB Client

Per interagire con il servizio DynamoDB si utilizza DynamoDbClient, specificando almeno:

- version: di default "latest"
- region: es. "eu-south-1"
- credentials: oggetto Credentials

```
$dynamodb = new
Aws\DynamoDb\DynamoDbClient([
    'version' => 'latest',
    'region' => 'eu-south-1',
    'credentials' => $credentials
]);
```

Metodi principali

createTable

- Crea una nuova tabella DynamoDB con attributi, chiavi primarie e parametri di throughput specificati.
- ⚠ La tabella potrebbe impiegare alcuni secondi per diventare disponibile dopo la creazione.

✂ Sintassi:

```
$result = $dynamodb->createTable([
    'TableName' => '<nome-tabella>', /*
        obbligatorio */
    'AttributeDefinitions' => [ /*
        obbligatorio */
        [
            'AttributeName' => '<nome-
                attributo>',
```

```
            'AttributeType' => 'S|N|B',
            // String | Number | Binary
        ],
        // ...
    ],
    'KeySchema' => [
        // obbligatorio
        [
            'AttributeName' => '<nome-
                attributo>',
            'KeyType' => 'HASH',
            // obbligatorio: chiave primaria
        ],
        [
            'AttributeName' => '<nome-
                attributo>',
            'KeyType' => 'RANGE',
            // opzionale: chiave secondaria
        ],
    ],
    'ProvisionedThroughput' => [
        // obbligatorio (a meno di on-demand)
        'ReadCapacityUnits' => 5,
        'WriteCapacityUnits' => 5,
    ]
]);
```

Descrizione dei parametri

Parametro	Descrizione
TableName	(Obbligatorio) Nome della tabella da creare.
Attribute-Definitions	(Obbligatorio) Lista degli attributi usati nelle chiavi della tabella. Ogni attributo necessita di un nome e un tipo (S, N, B).
KeySchema	(Obbligatorio) Definisce la chiave primaria (HASH) e, opzionalmente, la chiave secondaria (RANGE).
Provisioned-Throughput	(Obbligatorio) Specifica la capacità di lettura/scrittura (solo per modalità provisioned). Ignorare per tabelle on-demand.

💡 Esempio:

```
$dynamodb->createTable([
    'TableName' => 'utenti',
    'AttributeDefinitions' => [
        [
            'AttributeName' => 'user_id',
            'AttributeType' => 'S'
        ]
    ],
    'KeySchema' => [
        [
            'AttributeName' => 'user_id',
            'KeyType' => 'HASH'
        ]
    ],
    'ProvisionedThroughput' => [
        'ReadCapacityUnits' => 5,
        'WriteCapacityUnits' => 5
    ]
]);
```

📁 Questo comando crea una tabella utenti con chiave primaria **user_id** (di tipo stringa) e capacità di lettura/scrittura fissata a 5 unità.

💡 Esempio con altri attributi:

Supponiamo di voler creare una tabella utenti dove:

- user_id è la chiave primaria
- user_ip, email, created_at sono altri attributi

```
$result = $dynamoDb->createTable([
    'TableName' => 'utenti',
    'AttributeDefinitions' => [
        [
            'AttributeName' => 'user_id',
            'AttributeType' => 'S'
        ],
    ],
    'KeySchema' => [
        [
            'AttributeName' => 'user_id',
            'KeyType' => 'HASH'
        ],
    ],
    'ProvisionedThroughput' => [
        'ReadCapacityUnits' => 5,
        'WriteCapacityUnits' => 5,
    ],
]);
```

Poi potrai tranquillamente inserire record con altri attributi:

```
$dynamoDb->putItem([
    'TableName' => 'utenti',
    'Item' => [
        'user_id' => ['S' => '123'],
        'user_ip' => ['S' => '192.168.1.1'],
    ]
]);
```

```
        'email' => ['S' => 'test@example.com'],
        'created_at' => ['S' => '2025-07-21T12:00:00Z'],
    ]
]);
```

📘 Spiegazione tecnica

DynamoDB è **schema-less**, eccetto per le chiavi. Puoi aggiungere qualsiasi attributo nei record (Item) fintanto che la chiave primaria è rispettata.

❗ Importante

Se in futuro vorrai usare user_ip per creare un indice secondario (GSI o LSI), **allora sì**, dovrai dichiararlo esplicitamente nel AttributeDefinitions al momento della creazione della tabella (o dell'indice). Ma solo in quel caso.

📄 Output (semplificato):

```
[
    'TableDescription' => [
        'TableName' => 'utenti',
        'TableStatus' => 'CREATING',
        'TableArn' => '<arn>',
        // ...
    ]
]
```

Descrizione dei parametri dell'output

Parametro	Descrizione
TableName	Nome della tabella appena creata.
TableStatus	Stato attuale della tabella (CREATING, ACTIVE, ecc.).
TableArn	Identificatore univoco della tabella su AWS.

deleteTable

• Elimina in modo permanente una tabella da DynamoDB.

⚠ L'operazione è irreversibile e rimuove **tutti i dati** contenuti nella tabella.

✖ Sintassi:

```
$result = $dynamoDb->deleteTable([  
    'TableName' => '<nome-tabella>' /*  
    obbligatorio */  
]);
```

Descrizione dei parametri

Parametro	Descrizione
TableName	(Obbligatorio) Nome della tabella da eliminare.

💡 Esempio:

```
$dynamoDb->deleteTable([  
    'TableName' => 'Utenti'  
]);
```

📁 Questo comando elimina la tabella chiamata Utenti.

Output (semplificato):

```
[  
    'TableDescription' => [  
        'TableStatus' => 'DELETING',  
        // ... altri metadati della tabella  
    ],  
    'RequestId' => '<string>',  
]
```

Descrizione dei parametri dell'output

Parametro	Descrizione
TableDescription	Informazioni sulla tabella in fase di eliminazione.
TableStatus	Stato della tabella (tipicamente DELETING subito dopo la richiesta).
RequestId	ID univoco della richiesta per il tracciamento.

listTables

• Restituisce la lista delle tabelle DynamoDB presenti nell'account e nella regione configurata. Può essere usato per scorrere le tabelle esistenti, anche con paginazione.

✖ Sintassi:

```
$result = $dynamodb->listTables([
```

```
    'Limit' => <int>,           // opzionale:  
    massimo numero di tabelle da restituire
```

```
    'ExclusiveStartTableName' => '<string>'  
    // opzionale: nome da cui iniziare la  
    scansione per paginazione  
]);
```

Descrizione dei parametri

Parametro	Descrizione
Limit	Numero massimo di nomi di tabelle da restituire (default: 100).
ExclusiveStart- TableName	Nome tabella da cui riprendere la scansione (per paginazione).

💡 Esempio:

```
$result = $dynamodb->listTables([  
    'Limit' => 10  
]);
```

📁 Questo comando restituisce al massimo 10 nomi di tabelle.

Output (semplificato):

```
[
  'TableNames' => [
    '<nome-tabella-1>',
    '<nome-tabella-2>',
    // ...
  ],
  'LastEvaluatedTableName' =>
  '<string|null>'
]
```

Descrizione dei parametri dell'output

Parametro	Descrizione
TableNames	Array di stringhe con i nomi delle tabelle trovate.
LastEvaluated-TableName	Se presente, indica il nome della tabella da usare come cursore per paginare la lista successiva.

updateTable

• Modifica le impostazioni di una tabella esistente in DynamoDB.

⚠ Non consente di cambiare nome o tipo di chiavi primarie.

Sintassi:

```
$result = $dynamoDb->updateTable([
    'TableName' => '<nome-tabella>',
    // obbligatorio
```

```
'ProvisionedThroughput' => [
    // opzionale
    'ReadCapacityUnits' => <intero>,
    'WriteCapacityUnits' => <intero>
],
'StreamSpecification' => [
    // opzionale
    'StreamEnabled' => true|false,
    'StreamViewType' =>
    'NEW_IMAGE' | 'OLD_IMAGE' | 'NEW_AND_OLD_IMAGES' | 'KEYS_ONLY'
],
'AttributeDefinitions' => [
    /* obbligatorio se si
    modificano indici */
    [
        'AttributeName' =>
            '<nome-attributo>',
        'AttributeType' => 'S|N|B'
    ],
    ...
],
'GlobalSecondaryIndexUpdates' => [
    // opzionale
    [
        'Create' => [...],
        'Update' => [...],
        'Delete' => [...]
    ],
    ...
]
]);
```

Descrizione dei parametri principali

Parametro	Descrizione
TableName	(Obbligatorio) Nome della tabella da modificare.
Provisioned-Throughput	(Opzionale) Nuove capacità di lettura/scrittura. Necessario solo per modalità PROVISIONED.
Stream-Specification	(Opzionale) Abilita o disabilita lo stream DynamoDB e specifica il tipo di dati da includere.
Attribute-Definitions	(Obbligatorio solo se si modificano GSI) Specifica gli attributi usati negli indici.
Global-Secondary-IndexUpdates	(Opzionale) Permette la creazione, modifica o eliminazione di indici secondari globali.

Esempio:

```
$dynamoDb->updateTable([
    'TableName' => 'Utenti',
    'ProvisionedThroughput' => [
        'ReadCapacityUnits' => 10,
        'WriteCapacityUnits' => 5
    ]
]);
```

📁 Questo comando aggiorna la capacità di throughput della tabella Utenti.

Output (semplificato):

```
[
  'TableDescription' => [
    'TableStatus' => 'UPDATING',
    'ProvisionedThroughput' => [
      'ReadCapacityUnits' => 10,
      'WriteCapacityUnits' => 5,
      // ...
    ],
    // ... altri dettagli
  ],
  'RequestId' => '<string>',
]
```

Descrizione dei parametri dell'output

Parametro	Descrizione
TableDescription	Stato aggiornato della tabella.
TableStatus	Solitamente UPDATING durante l'operazione.
RequestId	ID univoco della richiesta.

describeTable

- Restituisce i metadati di una tabella DynamoDB specificata.
- 🔗 Utile per verificare schema, throughput, indici e stato attuale della tabella.

Sintassi:

```
$result = $dynamoDb->describeTable([
    'TableName' => '<nome-tabella>' //
    obbligatorio
]);
```

Descrizione dei parametri

Parametro Descrizione

TableName (Obbligatorio) Nome della tabella di cui ottenere i metadati.

Esempio:

```
$dynamoDb->describeTable([
    'TableName' => 'Utenti'
]);
```

📁 Questo comando recupera i metadati della tabella chiamata Utenti.

Output (semplificato):

```
[
  'Table' => [
    'TableName' => 'Utenti',
    'TableStatus' => 'ACTIVE',
    'CreationDateTime' => <DateTime>,
    'ItemCount' => 123,
```

```
    'TableSizeBytes' => 2048,
    'KeySchema' => [
      ['AttributeName' => 'id',
        'KeyType' => 'HASH'],
    ],
    'ProvisionedThroughput' => [
      'ReadCapacityUnits' => 5,
      'WriteCapacityUnits' => 5,
    ],
    // ... altri metadati
  ],
  'RequestId' => '<string>',
]
```

Descrizione dei parametri dell'output

Parametro	Descrizione
TableName	Nome della tabella.
TableStatus	Stato attuale della tabella (CREATING, UPDATING, ACTIVE, ecc.).
Creation-DateTime	Data e ora di creazione della tabella.
ItemCount	Numero stimato di item nella tabella.
TableSizeBytes	Dimensione stimata della tabella in byte.
KeySchema	Definizione delle chiavi primarie (HASH/optional RANGE).
Provisioned-Throughput	Capacità di lettura/scrittura provisionata, se non si usa on-demand.
RequestId	ID della richiesta, utile per debugging e tracciamento.



scan

- Recupera **tutti gli item** di una tabella DynamoDB o di un indice secondario globale (GSI).
- ⚠ Operazione costosa: legge ogni partizione → da usare con attenzione su tabelle grandi.



Sintassi:

```
$result = $dynamoDb->scan([  
    'TableName' => '<nome-tabella>',  
    /* obbligatorio */  
    'FilterExpression' => '<espressione>',  
    /* facoltativo */  
    'ExpressionAttributeValues' => [  
        /* richiesto se c'è filtro */  
        ':val1' => ['S' => 'valore']  
    ],  
    'Limit' => 100  
    /* facoltativo: max item da leggere */  
]);
```



Descrizione dei parametri

Parametro	Descrizione
TableName	(Obbligatorio) Nome della tabella da scansionare.
FilterExpression	(Facoltativo) Filtro per limitare i risultati.
Expression-AttributeValues	(Obbligatorio se usi un filtro) Valori associati agli alias usati nel filtro.
Limit	(Facoltativo) Numero massimo di item da restituire per chiamata.



Esempio:

```
$result = $dynamoDb->scan([  
    'TableName' => 'Utenti',  
    'FilterExpression' =>  
        'età >= :etaMin',  
    'ExpressionAttributeValues' => [  
        ':etaMin' => ['N' => '18']  
    ]  
]);
```

📁 Questo comando recupera tutti gli utenti con almeno 18 anni.



Output (semplificato):

```
[  
    'Items' => [ [...], [...], ... ],  
    'Count' => 100,  
    'ScannedCount' => 100,  
    'LastEvaluatedKey' => [...], /*  
        presente se ci sono altri item da  
        leggere */  
]
```



Descrizione dei parametri dell'output

Parametro	Descrizione
Items	Array degli item trovati.
Count	Numero di item restituiti.

Parametro	Descrizione
ScannedCount	Numero di item letti (anche quelli filtrati).
LastEvaluatedKey	Chiave dell'ultimo item letto, da usare per la paginazione.



Note:

- Usa LastEvaluatedKey per fare la paginazione e ottenere tutti i risultati.
- Per performance migliori, preferisci query() se conosci la chiave primaria.
- scan ignora gli indici locali (LSI).

putItem

- Inserisce o sovrascrive un elemento in una tabella DynamoDB.
- ⚠ Se l'elemento esiste già (stessa chiave primaria), viene sovrascritto.

✖ Sintassi:

```
$result = $dynamodb->putItem([
    'TableName'    =>    '<nome-tabella>',    //
    obbligatorio
    'Item' => [
        '<NomeAttributo>' =>
            ['<Tipo>' => '<Valore>'],
        // Altri attributi...
    ],
    //          'ConditionExpression'          =>
    'attribute_not_exists(ID)', // opzionale
    // 'ReturnConsumedCapacity' => 'TOTAL', //
    opzionale
]);
```

Descrizione dei parametri

Parametro	Descrizione
TableName	(Obbligatorio) Nome della tabella.
Item	(Obbligatorio) Mappa di attributi da salvare. Ogni attributo richiede un tipo (S, N, BOOL, ecc.) e un valore.
Condition-Expression	(Opzionale) Condizione da soddisfare prima di eseguire la scrittura. Utile per evitare sovrascritture involontarie.
Return-Consumed-Capacity	(Opzionale) Specifica se restituire info sul consumo di capacità (valori possibili: INDEXES, TOTAL, NONE).

💡 Esempio:

```
use Aws\DynamoDb\DynamoDbClient;

use Aws\Sdk;

$dynamodb = new DynamoDbClient([...]);

$dynamodb->putItem([
    'TableName' => 'Utenti',
    'Item' => [
        'ID' => ['N' => '1'],
        'Nome' => ['S' => 'Mario Rossi'],
        'user_ip' => ['S' => '192.168.1.100'],
        'is_admin' => ['BOOL' => false],
    ]
]);
```

📁 Questo comando crea o aggiorna un elemento con ID = 1 nella tabella Utenti, aggiungendo anche l'IP e un flag booleano.

📄 Output (semplificato):

```
[
    'ConsumedCapacity' => [
        'CapacityUnits' => float,
        'TableName' => string,
        ...
    ],
    ...
]
```

Descrizione dei parametri dell'output

Parametro	Descrizione
ConsumedCapacity	Info sul consumo di capacità, utile per analisi e ottimizzazione.

updateItem

• Aggiorna uno o più attributi di un item esistente in una tabella DynamoDB.

⚠ È necessario specificare la chiave primaria per identificare univocamente l'elemento da aggiornare.

✖ Sintassi:

```
$result = $dynamodb->updateItem([
    'TableName' => '<nome-tabella>',
    //obbligatorio
    'Key' => [ /* obbligatorio */
        '<nome-partition-key>' => [
            'S' => '<valore>'
        ],
        // opzionale se presente sort key
        '<nome-sort-key>' => [
            'N' => '<valore>'
        ]
    ],
    'UpdateExpression' => 'SET #attr = :val',
    'ExpressionAttributeNames' => [
        '#attr' => 'nome_attributo'
    ],
    'ExpressionAttributeValues' => [
        ':val' => ['S' => 'nuovo_valore']
    ],
]);
```

```
'ReturnValues' => 'UPDATED_NEW' // o
    'ALL_NEW', 'NONE', ecc.
]);
```



Descrizione dei parametri

Parametro	Descrizione
TableName	(Obbligatorio) Nome della tabella.
Key	(Obbligatorio) Chiave primaria dell'elemento da aggiornare. Deve includere partition key e, se presente, sort key.
Update-Expression	Espressione che definisce gli aggiornamenti da applicare agli attributi.
Expression-AttributeNames	Mappa dei nomi di attributo da usare nei placeholder (#). Necessario se il nome è una parola riservata.
Expression-AttributeValues	Valori da assegnare, associati ai placeholder (:).
ReturnValues	Specifica cosa restituire (opzionale).



Esempio:

```
$dynamodb->updateItem([
    'TableName' => 'utenti',
    'Key' => [
        'user_id' => ['S' => '123']
    ],
    'UpdateExpression' => 'SET #ip = :new_ip',
    'ExpressionAttributeNames' => [
```

```
    '#ip' => 'user_ip'
],
'ExpressionAttributeValues' => [
    ':new_ip' => ['S' => '192.168.0.42']
],
'ReturnValues' => 'UPDATED_NEW'
]);
```

📁 Questo comando aggiornerà l'attributo user_ip dell'utente con user_id = 123.

📄 Output (semplificato):

```
[
    'Attributes' => [
        'user_ip' => ['S' => '192.168.0.42']
    ],
    'RequestId' => '<string>'
]
```



Descrizione dei parametri dell'output

Parametro	Descrizione
Attributes	Attributi aggiornati, se specificato ReturnValues.
RequestId	ID della richiesta, utile per il debug.

getItem

• Recupera un singolo elemento da una tabella DynamoDB.

⚠ È necessario specificare l'intera chiave primaria dell'elemento (chiave partizionamento + opzionale chiave ordinamento).

Sintassi:

```
$result = $dynamodb->getItem([
    'TableName' => '<nome-tabella>',
    // obbligatorio

    'Key' => [
        // obbligatorio
        '<nome-partition-key>' => [
            'S' => '<valore>'
        ],
        // opzionale, se presente nella tabella
        '<nome-sort-key>' => ['N' =>
            '<valore>']
    ],
    'ProjectionExpression' =>
        '<lista-attributi>',
    // facoltativo

    'ConsistentRead' => true|false,
    // facoltativo
]);
```

Descrizione dei parametri	
Parametro	Descrizione
TableName	(Obbligatorio) Nome della tabella da cui leggere.
Key	(Obbligatorio) Chiave primaria dell'elemento da recuperare. Deve includere partition key e, se esiste, anche sort key.
ProjectionExpression	(Facoltativo) Stringa che definisce gli attributi da restituire.
ConsistentRead	(Facoltativo) Se true, esegue una lettura fortemente consistente (più lenta ma aggiornata).

Esempio:

```
$dynamodb->getItem([
    'TableName' => 'Utenti',
    'Key' => [
        'username' => [
            'S' => 'alice@example.com'
        ]
    ],
    'ProjectionExpression' =>
        'username, data_registrazione',
    'ConsistentRead' => true
]);
```

📁 Questo comando recupera l'utente con chiave username = 'alice@example.com' dalla tabella Utenti, restituendo solo username e data_registrazione.

Output (semplificato):

```
[
    'Item' => [
        'username' => [
            'S' => 'alice@example.com'
        ],
        'data_registrazione' => [
            'S' => '2024-01-01'
        ]
    ],
    'ConsumedCapacity' => [
        'CapacityUnits' => 0.5,
    ],
]
```

Descrizione dei parametri dell'output

Parametro	Descrizione
Item	Array associativo contenente gli attributi dell'elemento richiesto. Se l'elemento non esiste, Item non sarà presente.
Consumed-Capacity	Mostra le unità di capacità consumate dalla richiesta, se richiesto esplicitamente.



query

• Recupera uno o più elementi da una tabella DynamoDB usando la **chiave primaria** o un **indice secondario globale**.

🔍 Utilizza espressioni di condizione per filtrare i risultati.



Sintassi:

```
$result = $dynamoDb->query([
    'TableName' => '<nome-tabella>',
                    // obbligatorio
    'KeyConditionExpression' =>
        'chiave = :valore',
                    // obbligatorio
    'ExpressionAttributeValues' => [
        ':valore' => [
            'S' => '<valore-chiave>'
        ],
        // obbligatorio
    ],
    // Opzionali:
    'IndexName' => '<nome-indice-secondario>',
    'ProjectionExpression' => 'campo1, campo2',
    'FilterExpression' =>
        'campo2 = :altravalore',
    'ExpressionAttributeNames' =>
        ['#n' => 'nome'],
    'Limit' => 10,
    'ScanIndexForward' => false // ordinamento
                             discendente
]);
```



Descrizione dei parametri

Parametro	Descrizione
TableName	(Obbligatorio) Nome della tabella da interrogare.
KeyCondition-Expression	(Obbligatorio) Espressione per specificare la chiave primaria o la chiave dell'indice secondario.
Expression-AttributeValues	(Obbligatorio) Mappa dei valori da usare nelle espressioni.
IndexName	(Opzionale) Nome dell'indice secondario globale da usare, se diverso dalla chiave primaria.
Projection-Expression	(Opzionale) Specifica i campi da restituire (simile a SELECT).
FilterExpression	(Opzionale) Ulteriore filtro da applicare sui risultati dopo la KeyCondition.
Expression-AttributeNames	(Opzionale) Alias per i nomi di attributo, utile se contengono caratteri riservati.
Limit	(Opzionale) Numero massimo di elementi da restituire.
ScanIndexForward	(Opzionale) true per ordinamento crescente, false per discendente (default: true).



Esempio:

```
$dynamoDb->query([
    'TableName' => 'Utenti',
    'KeyConditionExpression' =>
        'user_id = :uid',
    'ExpressionAttributeValues' =>
```

```
[
    ':uid' => ['S' => '1234']
];
```

📁 Questo comando cerca tutti gli elementi della tabella Utenti con user_id = 1234.



Output (semplificato):

```
[
    'Items' => [
        [
            'user_id' => ['S' => '1234'],
            'nome' => ['S' => 'Mario'],
            // ... altri campi
        ],
        // altri elementi...
    ],
    'Count' => 1,
    'ScannedCount' => 1,
    'LastEvaluatedKey' =>
        [ /* per paginazione */ ],
    'RequestId' => '<string>',
]
```

Descrizione dei parametri dell'output

Parametro	Descrizione
Items	Array degli oggetti trovati.
Count	Numero di elementi restituiti.
ScannedCount	Numero di elementi analizzati internamente.
LastEvaluatedKey	Chiave da usare per ottenere la "pagina" successiva, se presente.
RequestId	ID univoco della richiesta, utile per log e debugging.

Differenze tra getItem e query in DynamoDB

Caratteristica	getItem	query
Obiettivo	Recuperare un singolo item esatto tramite chiave primaria.	Recuperare più item che condividono la stessa partition key.
Chiave richiesta	Richiede Partition Key + Sort Key (se presenti nella tabella).	Richiede almeno la Partition Key. Sort Key è opzionale ma può essere filtrata.
Risultato	Sempre un solo item (al massimo uno).	Più item (fino a 1 MB di dati o limite impostato).
Uso tipico	Accesso diretto e rapido a un item noto (lookup preciso).	Ricerca di un insieme di item correlati (es. log per utente, ordini per cliente, ecc.).
Prestazioni	Molto veloce, costo minimo.	Più costoso di getItem, ma comunque efficiente se ben indicizzato.
Supporta filtri	✗ No: recupera esattamente quello con la chiave specificata.	✓ Sì: supporta filtri su attributi, condizioni sulla sort key, proiezione di campi, ecc.

Caratteristica	getItem	query
Indice secondario	✗ Non può essere usato con indici secondari.	✓ Può usare anche indici secondari (GSI, LSI).

Quando usare getItem

- Quando conosci esattamente la chiave primaria (Partition Key + Sort Key, se presente).
- Per accessi rapidi e puntuali, simili a una lookup in una hash map.

Quando usare query

- Quando vuoi recuperare più item con la stessa Partition Key.
- Quando hai bisogno di filtrare i risultati su attributi o sulla Sort Key.
- Per lavorare con indici secondari.

deleteItem

- Elimina un singolo elemento da una tabella DynamoDB in base alla chiave primaria.

⚠ L'elemento viene rimosso definitivamente.

Sintassi:

```
$result = $dynamoDb->deleteItem([
    'TableName' => '<nome-tabella>',
    // obbligatorio
    'Key' => [
        // obbligatorio
        '<nome-partition-key>' => [
```

```
        'S' => '<valore>'
    ],
    // '<nome-sort-key>' => [
        'N' => '<valore>'
    ], // se presente
],
// 'ReturnValues' => 'ALL_OLD'
// opzionale
]);
```

Descrizione dei parametri

Parametro	Descrizione
TableName	(Obbligatorio) Nome della tabella.
Key	(Obbligatorio) Chiave primaria dell'elemento da eliminare.
ReturnValues	(Opzionale) Se impostato a 'ALL_OLD', restituisce i dati dell'elemento rimosso.

 Esempio:

```
$dynamoDb->deleteItem([
    'TableName' => 'Utenti',
    'Key' => [
        'email' => ['S' =>
            'utente@example.com']
    ],
    'ReturnValues' => 'ALL_OLD'
]);
```

 Questo comando elimina l'elemento con chiave email = 'utente@example.com' dalla tabella Utenti.

 Output (semplificato):

```
[
    'Attributes' => [
        'email' => ['S' =>
            'utente@example.com'],
        'nome' => ['S' => 'Mario']
        // altri attributi dell'elemento
        eliminato
    ],
    'RequestId' => '<string>',
]
```

 Descrizione dei parametri dell'output

Parametro	Descrizione
Attributes	Attributi dell'elemento rimosso (solo se ReturnValues è impostato a 'ALL_OLD').
RequestId	ID univoco della richiesta, utile per il tracciamento.